Watch 0 Fork 0

Debian packages build tool for native (arm64-arm64) or cross (amd64-arm64) build

BSD 3-Clause "New" or "Revised" License

Code of conduct

Contributing

Security policy

0 stars 0 forks 0 watching 1 branch 0 tags Activity

Public repository · Forked from qualcomm-linux/docker-pkg-build

1 Branch 0 Tags Go to file T Go to file Add file Code

This branch is 27 commits behind qualcomm-linux/docker-pkg-build:main . Contribute Sync fork

simonbeaudoin0935 Merge pull request qualcomm-linux#26 from qualcomm-linux/feat/tmp-hos...

108cdd3 · last month

.github	Debug docker push for RPM-based co...	3 months ago
Dockerfiles	Add rpmdevtools to Fedora containers	2 months ago
.gitignore	Initial commit	6 months ago
AGENTS.md	Prevent sbuild from auto-regenerating ...	2 months ago
CODE-OF-CONDUCT.md	Initial commit	6 months ago
CONTRIBUTING.md	Initial commit	6 months ago
LICENSE.txt	Initial commit	6 months ago
README.md	feat: add fixed /tmp mount flag	last month
SECURITY.md	Initial commit	6 months ago
color_logger.py	Initial commit	6 months ago
create_data_tar.py	Remove arch from create_data_tar.py	3 months ago
docker_deb_build.py	feat: add fixed /tmp mount flag	last month
docker_rpm_build.py	When no distro specified with --rebuild...	3 months ago

docker-pkg-build

A Python-based tool for building Debian packages inside Docker containers, designed for Qualcomm Linux projects. It supports ARM64 package builds across Ubuntu and Debian suites (`noble` , `questing` , `resolute` , `trixie` , and `sid`) to ensure consistent and reproducible package builds.

This repo encompases two use cases:

- Builder-agnostic local builds for a users wanting to build debian packages on their local machines in a repeatable way.
- Creating a docker image with required tooling and chroots for the different Qualcomm repo workflows.

See the **Github Workflow** section below.

Branches

main: Primary development branch. Contributors should develop submissions based on this branch and submit pull requests to this branch.

Requirements

- Python 3.6+
- Docker
- Git (for cloning repositories)

Installation Instructions

1. Clone the repository [in some tooling folder]:

```
git clone https://github.com/qualcomm-linux/docker-pkg-build.git
```

2. Ensure Docker is running and you have permissions to build containers. The build scripts does multiple pre-flight checks. Should any test fail, there will be instructions on what to do to fix the issue.

3. Pro tip: Say you cloned this to your home folder, create a quick alias in your .bashrc file (debb == **deb**ian **b**uild:

```
alias debb="~/docker-pkg-build/docker_deb_build.py"
```

First time using

Whenever you use the `docker_deb_build.py` script for the first time, it will need to build the containers. You can either defer this step to when you build your first package, or you can manually trigger the build of the containers as a confirmation step:

```
./docker_deb_build.py --rebuild
```

The build should take around ~15 min. After that, you can inspect the built images:

```
docker image ls
```

You should then see the following:

```
$ docker image ls
REPOSITORY                                                          TAG
IMAGE ID       CREATED             SIZE
ghcr.io/qualcomm-linux/pkg-builder                                  noble
bdbf1ec3b9bf   2 hours ago         1.25GB
ghcr.io/qualcomm-linux/pkg-builder                                  questing
ac7b0936a006   About an hour ago   1.32GB
ghcr.io/qualcomm-linux/pkg-builder                                  resolute
c41a1b076a1b   About an hour ago   1.35GB
ghcr.io/qualcomm-linux/pkg-builder                                  trixie
d00e2414b324   2 hours ago         1.47GB
ghcr.io/qualcomm-linux/pkg-builder                                  sid
8090ef2d71cc   About an hour ago   1.52GB
```

The image tag is the suite name. `docker_deb_build.py --distro <suite>` maps directly to the matching image tag `ghcr.io/qualcomm-linux/pkg-builder:<suite>` .

You can always build/rebuild the images with the command above.

Keeping builds fast

Internally, the docker_deb_build tool uses sbuild to build your package and needs a chroot to do that. Each of the containers contain one chroot. Every time sbuild is invoked, one of the first thing it does is issuing a apt-get update + upgrade inside the chroot to make sure you have the latest version of everything.

As time goes by and as new packages version come up compared to when you built the container, the apt update/upgrade step will take incrementally more time and basically repeat installing new versions eeeeevery time.

Therefore, in order to keep your build times minimal, you will want to rebuild your container periodically.

Every week or two is a good idea.

Building an hello-world example

You can test building the hello-world style pkg-example to prove everything works. Head over to the pkg-example page and have a look at the readme.

Then, clone and build :

```
alias debb=<docker-pkg-build location>/docker_deb_build.py

git clone git@github.com:qualcomm-linux/pkg-example.git
```

```
mkdir build

debb --source-dir pkg-example --output-dir build --distro questing
```

Usage

Run the `docker_deb_build.py` script to build Debian packages:

```
docker_deb_build.py --help
```

Key Features

- **Docker-based Builds**: Packages are built inside isolated Docker containers to ensure reproducibility.
- **Per-suite Builder Images**: Includes one Dockerfile and one prebuilt sbuild environment per supported suite.
- **Supported Suites**: Supports Ubuntu `noble`, `questing`, `resolute` and Debian `trixie`, `sid`.
- **Host-backed `/tmp` option for large builds**: `--mount-host-tmp` bind-mounts host `/tmp` to container `/tmp`.
- **Automated Workflows**: Integrates with GitHub Actions via the `qcom-container-build-and-upload.yml` workflow for CI/CD.

Host-backed `/tmp` for large package builds

Some large package builds need more host-backed temporary storage than the container default can provide.

Use `--mount-host-tmp` to bind-mount host `/tmp` to container `/tmp`. This is particularly relevant for large builds with multi-GB intermediate artifacts (for example, `pkg-camx`-sized builds around 20 GB).

For normal/smaller packages, this option is usually not necessary.

```
docker_deb_build.py \
  --source-dir pkg-camx \
  --output-dir build \
  --distro questing \
  --mount-host-tmp
```

Docker Images

To add a new suite, copy an existing suite Dockerfile in `Dockerfiles/` and adapt it for the new release. Also add the suite-specific Qualcomm source file under `Dockerfiles/sources/<suite>/qsc-deb-releases.sources`.

The last step is to ensure the new image is also pushed to GHCR as part of the `.github/workflows/qcom-container-build-and-upload.yml` workflow by adding a new line in the `Upload Debian Images` step:

```
docker push ghcr.io/${{env.QCOM_ORG_NAME}}/${{env.IMAGE_NAME}}:noble
docker push ghcr.io/${{env.QCOM_ORG_NAME}}/${{env.IMAGE_NAME}}:questing
docker push ghcr.io/${{env.QCOM_ORG_NAME}}/${{env.IMAGE_NAME}}:resolute
docker push ghcr.io/${{env.QCOM_ORG_NAME}}/${{env.IMAGE_NAME}}:trixie
docker push ghcr.io/${{env.QCOM_ORG_NAME}}/${{env.IMAGE_NAME}}:sid
# Add one more line for the new suite
```

GitHub Workflow

The repository includes a `qcom-container-build-and-upload.yml` workflow (located in `.github/workflows/`) that automates building and uploading Docker containers for package builds. This workflow is automatically executed every week so that the GHCR registry where the images are stored contains a one-week-or-less old image. This keeps build time as small as possible for workflows relying on those images. This is because when building using sbuild, the first step is doing an apt update; the older the image, the longer it takes doing this apt upgrade.

This also applies for non-github-workflow local builds; doing a **docker_deb_build.py --rebuild** periodically ensures a recent image and reduces the apt upgrade time at the start of every build.

Adding tooling

If additional tooling is required, the user shall add it to the **Dockerfiles/base-packages.txt**, open and merge the PR which will automatically trigger a post-merge build and upload to GHCR. Then, next time a github workflow build happens, the new tool will be present in the image hosted in GHCR.

How to enter the container

For whatever reason, you may have to enter the container in interactive mode. It could be testing installing extra tooling to make a build pass. Note: adapt the suite name and mounted paths for your scenario.

```
docker run --rm -it --privileged -v /local/mnt/workspace/sbeaudoi/extra-repo/libdmabufheap-1.0.r1.03200:/workspace/src:Z -v /local/mnt/workspace/sbeaudoi/extra-repo/build:/workspace/output:Z -w /workspace/src --name pkg-builder-questing ghcr.io/qualcomm-linux/pkg-builder:questing bash
```

Development

To contribute:

1. Fork the repository and create a feature branch from `main`.
2. Make your changes, ensuring tests pass.
3. Submit a pull request with a clear description of the changes.

See CONTRIBUTING.md for detailed guidelines.

Getting in Contact

- Report an Issue on GitHub
- Open a Discussion on GitHub
- E-mail me for general questions

License

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors

No contributors

Languages

Python 100%

Suggested workflows

Based on your tech stack

Publish Python Package
Publish a Python Package to PyPI on release.
By GitHub Actions
Configure

Python application
Create and test a Python application.
By GitHub Actions
Configure

Django
Build and Test a Django Project
By GitHub Actions
Configure

More workflows